



Exercise 4: Simulation of Dynamic Partition Allocation

Lappeenranta–Lahti University of Technology LUT

Computational Science and Artificial Intelligence

CT30A3350 Operating Systems and System Programming (Lahti)

Date: 15.04.2026

Md Hamidur Rahman Khan, 002475552

Examiner: Ankit Agrawal, PhD

The system starts with a single free memory partition of 640 KB. When a job requests memory, space is allocated from the lower address side of the chosen free partition. When memory is released, that partition is returned to the free list and any neighboring free partitions are merged. Two allocation methods were implemented in this task: first-fit and best-fit.

First-fit partition allocation algorithm result

```
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex4$ gcc -o dynamic_partition dynamic_partition.c
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex4$ ./dynamic_partition

===== First-Fit =====
Initial state:
Free list:
  Block 1 -> start = 0 KB, size = 640 KB

Step 1: job1 applies for 130 KB
job1 allocates 130 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 510 KB

Step 2: job2 applies for 60 KB
job2 allocates 60 KB successfully
Free list:
  Block 1 -> start = 190 KB, size = 450 KB

Step 3: job3 applies for 100 KB
job3 allocates 100 KB successfully
Free list:
  Block 1 -> start = 290 KB, size = 350 KB

Step 4: job2 releases 60 KB
job2 releases 60 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 60 KB
  Block 2 -> start = 290 KB, size = 350 KB

Step 5: job4 applies for 200 KB
job4 allocates 200 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 60 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 6: job3 releases 100 KB
job3 releases 100 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 160 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 7: job1 releases 130 KB
job1 releases 130 KB successfully
```

Figure 1. Terminal showing compile and run command with the resulting output of the first-fit partition allocation algorithm (part 1)

```

Step 7: job1 releases 130 KB
job1 releases 130 KB successfully
Free list:
  Block 1 -> start = 0 KB, size = 290 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 8: job5 applies for 140 KB
job5 allocates 140 KB successfully
Free list:
  Block 1 -> start = 140 KB, size = 150 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 9: job6 applies for 60 KB
job6 allocates 60 KB successfully
Free list:
  Block 1 -> start = 200 KB, size = 90 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 10: job7 applies for 50 KB
job7 allocates 50 KB successfully
Free list:
  Block 1 -> start = 250 KB, size = 40 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 11: job6 releases 60 KB
job6 releases 60 KB successfully
Free list:
  Block 1 -> start = 140 KB, size = 60 KB
  Block 2 -> start = 250 KB, size = 40 KB
  Block 3 -> start = 490 KB, size = 150 KB

```

Figure 2. Terminal showing the resulting output of the first-fit partition allocation algorithm (part 2)

In the first-fit algorithm, the system scans the free partition list from low address to high address and allocates the first partition large enough for the request. This method is simple and fast. In this experiment, the algorithm began to differ from best-fit when job5 requested 140 KB. First-fit selected the free partition starting at address 0 because it was the first partition large enough.

After the full request sequence, the final free partition list for first-fit was:

- start = 140 KB, size = 60 KB
- start = 250 KB, size = 40 KB
- start = 490 KB, size = 150 KB

Best-fit partition allocation algorithm result

```
===== Best-Fit =====
Initial state:
Free list:
  Block 1 -> start = 0 KB, size = 640 KB

Step 1: job1 applies for 130 KB
job1 allocates 130 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 510 KB

Step 2: job2 applies for 60 KB
job2 allocates 60 KB successfully
Free list:
  Block 1 -> start = 190 KB, size = 450 KB

Step 3: job3 applies for 100 KB
job3 allocates 100 KB successfully
Free list:
  Block 1 -> start = 290 KB, size = 350 KB

Step 4: job2 releases 60 KB
job2 releases 60 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 60 KB
  Block 2 -> start = 290 KB, size = 350 KB

Step 5: job4 applies for 200 KB
job4 allocates 200 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 60 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 6: job3 releases 100 KB
job3 releases 100 KB successfully
Free list:
  Block 1 -> start = 130 KB, size = 160 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 7: job1 releases 130 KB
job1 releases 130 KB successfully
Free list:
  Block 1 -> start = 0 KB, size = 290 KB
  Block 2 -> start = 490 KB, size = 150 KB
```

Figure 3. The resulting output of the best-fit partition allocation algorithm (part 1)

```

Step 7: job1 releases 130 KB
job1 releases 130 KB successfully
Free list:
  Block 1 -> start = 0 KB, size = 290 KB
  Block 2 -> start = 490 KB, size = 150 KB

Step 8: job5 applies for 140 KB
job5 allocates 140 KB successfully
Free list:
  Block 1 -> start = 0 KB, size = 290 KB
  Block 2 -> start = 630 KB, size = 10 KB

Step 9: job6 applies for 60 KB
job6 allocates 60 KB successfully
Free list:
  Block 1 -> start = 60 KB, size = 230 KB
  Block 2 -> start = 630 KB, size = 10 KB

Step 10: job7 applies for 50 KB
job7 allocates 50 KB successfully
Free list:
  Block 1 -> start = 110 KB, size = 180 KB
  Block 2 -> start = 630 KB, size = 10 KB

Step 11: job6 releases 60 KB
job6 releases 60 KB successfully
Free list:
  Block 1 -> start = 0 KB, size = 60 KB
  Block 2 -> start = 110 KB, size = 180 KB
  Block 3 -> start = 630 KB, size = 10 KB
hamidur@DESKTOP-7CGRQ8S:~/LUT_exercises/os_sp/ex4$

```

Figure 4. The resulting output of the best-fit partition allocation algorithm (part 2)

In the best-fit algorithm, the system chooses the smallest free partition that can still satisfy the request. So, the remaining unused space is usually smaller. In this experiment, the difference appeared when job5 requested 140 KB. Instead of choosing the 290 KB partition at address 0, best-fit selected the 150 KB partition at address 490, since it was the smallest partition that could fit the job.

After the full request sequence, the final free partition list for best-fit was:

- start = 0 KB, size = 60 KB
- start = 110 KB, size = 180 KB
- start = 630 KB, size = 10 KB

Analysis

At the beginning of the request sequence, first-fit and best-fit gave the same result. This was because there was either only one partition that could satisfy the request, or both methods ended up choosing the same one. The difference appeared once there were several possible free partitions. First-fit selected the first block that was large enough based on address order, while best-fit selected the smallest block that could still satisfy the request. Because of this, the pattern of fragmentation became different. First-fit left a larger continuous free space in the higher address region, while best-fit created a very small 10 KB fragment after allocating job5.

This simulation shows that both first-fit and best-fit can handle dynamic partition allocation and recycling correctly, but they organize free space differently. First-fit is simpler and faster because it stops as soon as it finds a suitable block. Best-fit uses space more tightly, but it can also leave behind very small fragments that are harder to use later.